

**STREDNÁ PRIEMYSELNÁ ŠKOLA JOZEFA MURGAŠA
BANSKÁ BYSTRICA**



BLOCKSENSE DIGITÁLNA PEŇAŽENKA

MATURITNÁ PRÁCA PČOZ forma b)

MAROŠ ĎURIŠ, IV. A

BANSKÁ BYSTRICA 2025/2026

**STREDNÁ PRIEMYSELNÁ ŠKOLA JOZEFA MURGAŠA,
HURBANOVA 6, BANSKÁ BYSTRICA**

Meno a priezvisko:	Maroš Ďuriš
Trieda:	IV. A
Školský rok:	2025/2026
Odbor:	2561 M informačné a sieťové technológie
Názov práce:	Digitálna peňaženka pre správu kryptomien
Zadanie práce:	Navrhnuť a implementovať registráciu a prihlasovanie používateľov Vytvoriť bezpečný systém autentifikácie a autorizácie používateľov Navrhnuť správu používateľov a prístupových práv podľa rolí Zabezpečiť ochranu a bezpečné ukladanie citlivých údajov Navrhnuť prehľadné a intuitívne používateľské rozhranie Naštudovať princípy blockchain technológií v prostredí .NET Implementovať správu kryptomenových peňaženiek v aplikácii Vypracovať sprievodnú technickú dokumentáciu
Dátum zadania:	01. 10. 2025
Konzultant:	Mgr. Horalová Kalinová Michaela, PhD.
Podpis konzultanta:	

ČESTNÉ VYHLÁSENIE

Čestne vyhlasujem, že túto maturitnú prácu som vypracoval samostatne s použitím uvedených literárnych zdrojov. Som si vedomý dôsledkov, ak uvedené údaje nie sú pravdivé.

Banská Bystrica 02. 03. 2026

.....

vlastnoručný podpis autora

OBSAH

ZOZNAM SKRATIEK, ZNAČIEK A SYMBOLOV	7
ZOZNAM OBRÁZKOV, TABULIEK A GRAFOV	8
ÚVOD	9
1 TEORETICKÉ VÝCHODISKÁ.....	10
1.1 .NET.....	10
1.2 ASP.NET Core	10
1.3 Avalonia.....	10
1.4 MySQL	11
1.5 API.....	11
1.6 OAuth	11
1.7 Git	11
1.8 BouncyCastle	12
1.9 Blockchain.....	12
2 CIELE PRÁCE.....	13
3 MATERIÁL A METODIKA PRÁCE.....	14
3.1 Výber technológií a nástrojov	14
3.1.1 Desktopový klient – Avalonia UI Framework.....	14
3.1.2 Backend – ASP.NET Core Web API	14
3.1.3 Databázové riešenie.....	15
3.1.4 Bezpečnostné mechanizmy a zoznam použitých balíčkov.....	15
3.1.5 Verzovanie kódu – Git.....	16
3.2 Návrh architektúry systému.....	16
3.2.1 Databázová vrstva.....	16
3.2.2 Repozitárová vrstva.....	17
3.2.3 Servisná vrstva.....	17

3.2.4	<i>Controller vrstva</i>	17
3.2.5	<i>Komunikácia medzi klientom a serverom</i>	18
3.2.6	<i>Pozvánkový systém</i>	18
3.2.7	<i>Spracovanie chýb a výnimiek na serveri</i>	18
3.3	Implementácia autentifikácie a autorizácie	18
3.3.1	<i>Registrácia používateľa</i>	19
3.3.2	<i>Autentifikácia používateľa</i>	19
3.3.3	<i>Správa tokenov</i>	20
3.3.3.1	<i>Access Token</i>	20
3.3.3.2	<i>Refresh token</i>	20
3.3.4	<i>Dvojfaktorová autentifikácia (2FA)</i>	21
3.3.4.1	<i>Inicializácia</i>	21
3.3.4.2	<i>Overenie</i>	21
3.3.4.3	<i>Záložné kódy</i>	21
3.4	Vytváranie kryptomenovej peňaženky	22
3.4.1	<i>Generovanie mnemonic frázy a seedu</i>	22
3.4.2	<i>Inicializácia peňaženky a správa adresy</i>	22
3.4.3	<i>Import a export peňaženky</i>	23
3.4.4	<i>Aktívna peňaženka</i>	24
3.5	Správa peňaženky a integrácia rôznych kryptomien	24
3.5.1	<i>Synchronizácia zostatku a transakcií</i>	24
3.5.2	<i>Realizácia transakcií</i>	24
3.5.3	<i>Podpora viacerých kryptomien</i>	25
4	VÝSLEDKY PRÁCE A DISKUSIA	26
4.1	Splnenie stanovených cieľov	26
4.2	Testovanie a overenie funkčnosti	26
4.3	Možnosti ďalšieho rozvoja	26

5 ZÁVER PRÁCE	27
ZHRNUTIE	28
ZOZNAM POUŽITEJ LITERATÚRY	29
ZOZNAM PRÍLOH.....	31
PRÍLOHY	32

ZOZNAM SKRATIEK, ZNAČIEK A SYMBOLOV

AES	Advanced Encryption Standard
API	Application Programming Interface
ASP	Active Server Pages
BIP	Bitcoin Improvement Proposal
DB	Database
GC	Garbage Collection
HD	Hierarchically Deterministic
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IP	Internet Protocol
JWT	JSON Web Token
JSON	JavaScript Object Notation
MVC	Model-View-Controller
OAuth	Open Authorization
OS	Operating System
QR	Quick Response
REST	Representational State Transfer
RSA	Rivest–Shamir–Adleman
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TOTP	Time-based One-Time Password
UI	User Interface
URI	Uniform Resource Identifier
2FA	Two-Factor Authentication

ZOZNAM OBRÁZKOV, TABULIEK A GRAFOV

Obrázok 1 Architektúra viacvrstvovej aplikácie (MVC) (10)	16
Obrázok 2 Sekvenčný diagram autorizačného toku OAuth 2.0 (11).....	20
Obrázok 3 Diagram procesu inicializácie TOTP (12)	21
Obrázok 4 Proces generovania mnemonickej frázy (BIP39) (13).....	22
Obrázok 5 Hierarchicky deterministické peňaženky (BIP32) (14)	23

ÚVOD

Rozhodli sme sa pre vývoj softvérového riešenia desktopovej kryptopeňaženky s cieľom rozvíjať zručnosti vo vývoji bezpečných aplikácií, kryptografii a spoľahlivom spracovaní citlivých dát. Projekt sa zameriaval na vytvorenie systému umožňujúceho bezpečnú správu kryptomien a používateľských účtov, pričom kládol dôraz na integritu dát, stabilitu a efektívnosť prevádzky.

Práca pozostávala z klienta a backendového API servera. Pri návrhu sme sa sústredili na bezpečnosť a spoľahlivosť, pričom sme využili moderné šifrovacie mechanizmy a osvedčené postupy ochrany citlivých informácií. Rovnako sme sa zamerali na stabilitu a efektívnosť, aby aplikácia konzistentne vykonávala všetky operácie.

Používateľské rozhranie bolo navrhnuté s ohľadom na intuitívnosť, prehľadnosť a jednoduchosť, čím sme zjednodušili používanie aplikácie aj pre menej skúsených používateľov. Projekt bol verzovaný a spravovaný prostredníctvom Git-u, čo umožnilo efektívne sledovanie zmien, prácu s viacerými vývojovými vetvami a bezpečné nasadzovanie nových funkcií.

Dokumentácia podrobne opisuje architektúru systému, implementované funkcie, bezpečnostné opatrenia a postupy, ktoré zabezpečujú bezpečný a efektívny chod systému, vrátane odporúčaných postupov pri nasadení a údržbe softvérového riešenia v reálnom prostredí.

1 TEORETICKÉ VÝCHODISKÁ

1.1 .NET

.NET je bezplatná, multiplatformová, open-source vývojová platforma na tvorbu rôznych druhov aplikácií. Dokáže spúšťať programy napísané v rôznych jazykoch, pričom C# je najpopulárnejší. Spolieha sa na vysoko výkonný runtime, ktorý je používaný v produkcii mnohými aplikáciami s veľkým zaťažením.

Platforma .NET bola navrhnutá tak, aby poskytovala produktivitu, výkon, bezpečnosť a spoľahlivosť. Poskytuje automatickú správu pamäte prostredníctvom garbage collectoru. Je typovo bezpečná a pamäťovo bezpečná, vďaka používaniu GC a prísnyh kompilátorom jazyka. Ponúka konkurentnosť prostredníctvom async/await a Task primitív. Obsahuje veľkú sadu knižníc, ktoré majú širokú funkcionálnosť a boli optimalizované pre výkon na viacerých operačných systémoch a architektúrach čipov. (1)

1.2 ASP.NET Core

ASP.NET Core je open-source modulárny rámec pre webové aplikácie. Je to prepracovanie ASP.NET, ktoré spája predtým oddelené ASP.NET MVC a ASP.NET Web API do jedného programovacieho modelu. Napriek tomu, že ide o nový rámec, postavený na novom webovom stacku, má vysokú mieru konceptuálnej kompatibility s ASP.NET. Rámec ASP.NET Core podporuje vedľa seba viacero verzií, takže rôzne aplikácie vyvíjané na jednom počítači môžu cieľiť na rôzne verzie ASP.NET Core. To nebolo možné s predchádzajúcimi verziami ASP.NET. ASP.NET Core pôvodne bežal na Windows-only .NET Framework aj na multiplatformovom .NET. Podpora pre .NET Framework však bola odstránená začínajúc ASP.NET Core 3.0. (2)

1.3 Avalonia

Avalonia je open-source, multiplatformový UI framework, ktorý umožňuje vývojárom vytvárať aplikácie pomocou .NET pre Windows, macOS, Linux, iOS, Android a WebAssembly.

Používa vlastný rendering engine na vykresľovanie UI ovládacích prvkov, čím zabezpečuje konzistentný vzhľad a správanie naprieč všetkými podporovanými platformami. To znamená, že vývojári môžu zdieľať svoj UI kód a zachovať jednotný vzhľad a pocit bez ohľadu na cieľovú platformu. (3)

1.4 MySQL

MySQL, najpopulárnejší open source SQL systém na správu databáz, je vyvíjaný, distribuovaný a podporovaný spoločnosťou Oracle Corporation. Databáza je štruktúrovaná kolekcia údajov. Môže ísť o čokoľvek, od jednoduchého nákupného zoznamu až po galériu obrázkov alebo obrovské množstvo informácií v podnikovej sieti. Na pridávanie, prístup a spracovanie údajov uložených v počítačovej databáze potrebujete systém na správu databáz, ako je MySQL Server.

MySQL databázy sú relačné. Relačná databáza ukladá údaje do samostatných tabuliek, namiesto toho, aby všetky údaje boli uložené v jednom veľkom úložisku. Štruktúry databáz sú organizované do fyzických súborov optimalizovaných pre rýchlosť. Logický model, s objektmi ako databázy, tabuľky, pohľady, riadky a stĺpce, poskytuje flexibilné programovacie prostredie. (4)

1.5 API

Application programming interface alebo skratkou API (rozhranie pre programovanie aplikácií). Tento termín je používaný v programovaní. Ide o zbierku funkcií a tried (ale aj iných programov), ktoré určujú akým spôsobom sa majú funkcie knižníc volať zo zdrojového kódu programu. API funkcie sú programové celky, ktoré programátor volá namiesto vlastného naprogramovania. Rozhranie knižnice, ktorá sa používa pri prekladaní programu do binárnej podoby sa nazýva ABI. Grafické knižnice ako OpenGL alebo DirectX majú taktiež zabudované vlastné API funkcie. (5)

1.6 OAuth

OAuth 2.0 je priemyselný štandardný protokol pre autorizáciu. OAuth 2.0 sa zameriava na jednoduchosť pre vývojárov klientskych aplikácií, pričom poskytuje špecifické autorizačné toky pre webové aplikácie, desktopové aplikácie, mobilné telefóny a zariadenia do obývačky. Táto špecifikácia a jej rozšírenia sú vyvíjané v rámci pracovnej skupiny IETF OAuth. (6)

1.7 Git

Git je distribuovaný softvérový systém pre správu verzií, ktorý je schopný spravovať verzie zdrojového kódu alebo dát. Často sa používa na kontrolu zdrojového kódu programátormi, ktorí vyvíjajú softvér spoluprácou.

Ciele návrhu Git zahŕňajú rýchlosť, integritu dát a podporu distribuovaných, nelineárnych pracovných tokov – tisíce paralelných vetiev bežiacich na rôznych počítačoch. Rovnako ako väčšina iných distribuovaných systémov pre správu verzií, a na rozdiel od väčšiny klient-server systémov, Git uchováva lokálnu kópiu celého repozitára, známeho aj ako „repo“, s históriou a schopnosťami sledovania verzií, nezávisle od prístupu k sieti alebo centrálnemu serveru. Repozitár je uložený na každom počítači v štandardnom adresári s doplnkovými, skrytými súbormi, ktoré poskytujú schopnosti správy verzií. (7)

1.8 BouncyCastle

Projekt Bouncy Castle ponúka open-source API pre Java, C# a Kotlin, ktoré podporujú kryptografiu a kryptografické protokoly. Pokrývajú mnoho oblastí bezpečnosti, ako sú infraštruktúra verejných kľúčov, digitálne podpisy, autentifikácia a bezpečná komunikácia. Okrem toho sú k dispozícii aj FIPS certifikované verzie pre Java aj C#. (8)

1.9 Blockchain

Blockchain je zdieľaná, nemenná digitálna účtovná kniha, ktorá umožňuje zaznamenávanie transakcií a sledovanie aktív v sieti. Funguje ako decentralizovaná distribuovaná databáza, kde sú dáta uložené na viacerých počítačoch, čo ho robí odolným proti manipulácii. Transakcie sú zoskupené do blokov, ktoré sú prepojené do bezpečného a transparentného reťazca, čo zaručuje integritu údajov. Blockchain poskytuje bezpečnosť, transparentnosť a dôveru bez spoliehania sa na tradičných sprostredkovateľov a pomáha firmám zlepšiť efektivitu a znížiť náklady. Táto technológia nachádza uplatnenie nielen v kryptomenách, ale aj v správe dodávateľských reťazcov, zdravotníctve a finančných službách. Okrem toho umožňuje sledovanie a auditovanie transakcií v reálnom čase, čo zvyšuje zodpovednosť a dôveru v systéme. (9)

2 CIELE PRÁCE

Hlavný cieľ:

Hlavným cieľom projektu bolo navrhnúť a implementovať digitálnu peňaženku vo forme desktopovej aplikácie, určenej najmä na správu kryptomien. Aplikácia bola vyvinutá pomocou technológií .NET (C#), ASP.NET Core Web API a MySQL, pričom používateľské rozhranie bolo realizované prostredníctvom Avalonia UI Frameworku. Výstupom práce bol ucelený a plne funkčný návrh desktopovej aplikácie, ktorá umožnila používateľovi bezpečne spravovať vlastné prostriedky a interagovať s blockchain sieťami.

Čiastkové ciele:

- Zabezpečiť možnosť registrácie a prihlasovania používateľov s overením oprávnenia na prístup do systému.
- Vytvoriť bezpečný systém autentifikácie a autorizácie používateľov s podporou viacúrovňovej ochrany účtu.
- Navrhnúť a implementovať správu používateľov a prístupových práv podľa ich rolí v systéme.
- Umožniť používateľom vytvárať a spravovať vlastné kryptomenové peňaženky v rámci aplikácie.
- Zabezpečiť bezpečné ukladanie a ochranu citlivých údajov, vrátane privátnych kľúčov a seed fráz.
- Implementovať funkcionality pre realizáciu kryptomenových transakcií a ich prehľadné zobrazenie v histórii.
- Vytvoriť prehľadné a intuitívne používateľské rozhranie, ktoré umožní jednoduchú správu účtu, transakcií a bezpečnostných nastavení.
- Naštudovať princípy blockchain technológií, bezpečnostných mechanizmov a možnosti ich implementácie v prostredí .NET.
- Vypracovať sprievodnú dokumentáciu, ktorá popisuje architektúru riešenia, použité technológie a bezpečnostné postupy.
- Doložiť priebežnú vizualizáciu riešenia vrátane návrhov používateľského rozhrania a ukážok implementovaných funkcionalít.

3 MATERIÁL A METODIKA PRÁCE

3.1 Výber technológií a nástrojov

Pri realizácii projektu BlockSense sme zvolili technológie podľa troch hlavných kritérií: bezpečnosť pri práci s citlivými kryptografickými údajmi, výkon pri spracovaní požiadaviek a dlhodobá udržateľnosť kódu. Každú technológiu sme pred zahrnutím do projektu overili z hľadiska aktívnej podpory, dostupnosti bezpečnostných protokolov a kompatibility s ostatnými zvolenými nástrojmi.

3.1.1 Desktopový klient – Avalonia UI Framework

Pre vývoj desktopového klienta sme zvolili Avalonia UI Framework. Tento framework sme uprednostnili pred alternatívami, pretože umožňuje zdieľať kód používateľského rozhrania naprieč platformami Windows, macOS a Linux bez nutnosti písať platformovo špecifické implementácie. Hoci sme sa primárne zamerali na Windows ako hlavný cieľový operačný systém, multiplatformová podpora nám ponúkala priestor na prípadné rozšírenie bez zásahov do existujúceho kódu aplikácie.

Pri návrhu používateľského rozhrania sme kládli dôraz na prehľadnosť a intuitívnosť. Postupovali sme modulárne – každú funkčnú oblasť, napríklad správu peňaženiek, históriu transakcií alebo nastavenia zabezpečenia, sme implementovali ako samostatný komponent alebo službu. Tento prístup nám umožnil testovať a upravovať jednotlivé časti rozhrania nezávisle od zvyšku aplikácie.

3.1.2 Backend – ASP.NET Core Web API

Backend sme implementovali pomocou ASP.NET Core Web API. Zvolili sme RESTful architektúru, kde každý endpoint zodpovedal konkrétnej operácii: registrácii, prihlasovaniu, správe tokenov, synchronizácii zostatku alebo odosielaniu transakcií. Pre správu relácií a autentifikáciu sme využili protokol OAuth 2.0, ktorý nám umožnil oddeliť proces vydávania tokenov od samotnej aplikačnej logiky.

Architektúru servera sme navrhli do vrstiev, pričom každá vrstva mala presnú zodpovednosť. Controller vrstva prijímala a validovala požiadavky, servisná vrstva obsahovala samotnú logiku a repozitárová vrstva komunikovala s databázou. Toto striktné oddelenie zodpovedností nám zjednodušilo ladenie chýb a pridávanie nových funkcionalít.

3.1.3 Databázové riešenie

Databázová časť projektu bola rozdelená na dve časti podľa charakteru ukladaných dát. Na strane servera sme použili relačnú databázu MySQL, do ktorej sme ukladali vzdialené používateľské dáta, ako napríklad záznamy o účtoch, pozývacích kódoch a aktivitách používateľov. Na strane klienta sme použili lokálnu databázu LevelDB, do ktorej sme ukladali šifrované seed frázy a privátne kľúče, čím sme zabezpečili rýchly prístup k citlivým dátam bez ich odosielania na server.

Táto kombinácia serverovej a lokálnej databázy nám umožnila dôsledne uplatniť princíp zero-knowledge: server v žiadnom okamihu neobsahuje ani nevidí privátne kryptografické materiály používateľa. Všetky operácie vyžadujúce privátny kľúč prebiehali výhradne na strane klienta, v operačnej pamäti, bez akéhokoľvek prenosu na server.

Pri práci s MySQL sme dôsledne používali databázové transakcie. Každú operáciu zahŕňajúcu viacero krokov, napríklad registráciu používateľa spojenú s označením pozývacieho kódu ako použitého, sme vykonávali v rámci transakcie. V prípade zlyhania ktoréhokoľvek kroku sme vykonali rollback, teda návrat do pôvodného stavu, čím sme zaručili konzistentný stav databázy.

3.1.4 Bezpečnostné mechanizmy a zoznam použitých balíčkov

Na ochranu citlivých údajov sme implementovali AES-256 šifrovanie pre ukladanie seed fráz a privátnych kľúčov a Argon2Id hashovanie pre bezpečné ukladanie hesiel. Na implementáciu týchto mechanizmov sme využili nasledujúce NuGet balíčky:

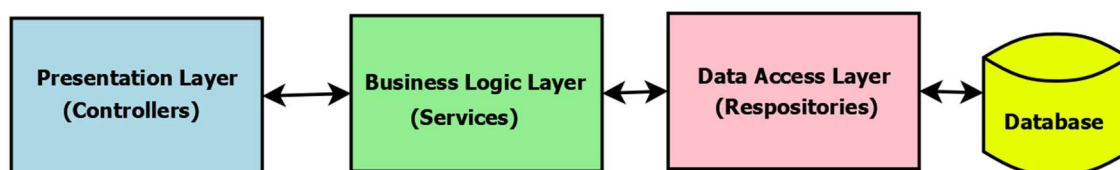
- BouncyCastle.Cryptography (2.6.2) – poskytuje kryptografické algoritmy, vrátane AES, RSA, HMAC a generovania kľúčov.
- Microsoft.AspNetCore.Authentication.JwtBearer (8.0.23) – implementácia JWT autentifikácie na serveri.
- Otp.NET (1.4.1) – generovanie TOTP kódov pre dvojfaktorovú autentifikáciu.
- MySql.Data (9.6.0) – klient pre pripojenie ASP.NET Core API k MySQL databáze.
- Dapper (2.1.66) – mikro-ORM pre efektívne mapovanie výsledkov MySQL dotazov na entity a jednoduchú prácu s databázou.
- Serilog (3.0.1) – logovanie aktivít a chýb.

3.1.5 Verzovanie kódu – Git

Dôležitou súčasťou vývoja bola správa kódu prostredníctvom systému Git, ktorý nám umožnil sledovanie histórie zmien, bezpečné riadenie vývojových vetiev a efektívnu implementáciu jednotlivých funkcionalít. Každá nová funkcia bola implementovaná vo vlastnej vetve a po otestovaní zlúčená do hlavnej vetvy. Správy commitov sme písali so stručným popisom uskutočnenej zmeny, čím sme jednoznačne identifikovali, v ktorom bode vývoja bola konkrétna funkcionálna zavedená alebo upravená.

3.2 Návrh architektúry systému

Pri návrhu systému BlockSense sme sa rozhodli použiť klient-server architektúru, ktorá jasne oddelila používateľské rozhranie od aplikačnej logiky a spracovania citlivých dát. Toto oddelenie nám umožnilo nezávisle a efektívne vyvíjať a testovať klientsku a serverovú časť, pričom ich komunikácia prebiehala výhradne cez definované REST API rozhranie.



Obrázok 1 Architektúra viacvrstvovej aplikácie (MVC) (10)

3.2.1 Databázová vrstva

V serverovej databáze MySQL sme udržiavali záznamy o používateľských účtoch, pozývacích kódoch a aktivitách používateľov. Každé heslo sme pred uložením hashovali algoritmom Argon2id s náhodne vygenerovaným saltom, ktorý sme uložili spolu s hashom do samostatného stĺpca. Tento prístup zabezpečil, že aj v prípade úniku databázy nebolo možné heslá priamo zneužiť.

Refresh tokeny sme pred zápisom do databázy hashovali algoritmom SHA-256, čím sme zaručili, že ani správca databázy nemal prístup k ich pôvodnej hodnote. Ku každému záznamu refresh tokenu sme uložili presný dátum a čas vydania a expirácie, ktorý server kontroloval pri každej žiadosti o obnovenie access tokenu.

Pre dvojfaktorovú autentifikáciu sme v databáze uchovávali šifrovaný TOTP tajný kľúč a hashované záložné kódy. Každý záložný kód sme po použití označili ako spotrebovaný, čím sme zamedzili jeho opakovanému využitiu. Záznamy o aktivitách

používateľov, ako napríklad čas prihlásenia, IP adresa a identifikátor zariadenia, sme ukladali do samostatnej tabuľky slúžiacej na detekciu podozrivých prístupov a prípadnú revokáciu tokenov.

3.2.2 Repozitárová vrstva

Repozitárovú vrstvu sme implementovali ako samostatnú dátovú prístupovú vrstvu. Pre každú entitu, teda pre používateľa, token, transakciu aj pozvánku, sme vytvorili samostatný repozitár s metódami na vytvorenie, načítanie, aktualizáciu a odstránenie záznamov. Na formulovanie SQL dotazov a mapovanie výsledkov na doménové objekty sme použili knižnicu Dapper, čo nám dalo plnú kontrolu nad SQL bez nutnosti písať rozsiahly kód mapujúci entity.

3.2.3 Servisná vrstva

Servisná vrstva tvorila jadro systému. Pri autentifikácii sme po prijatí prihlasovacích údajov overili hash hesla uložený v databáze, vygenerovali JWT access token a vytvorili nový refresh token s definovanou expiráciou. Pri autorizácii sme z JWT tokenu načítali rolu používateľa a podľa nej povolili alebo zamietli konkrétnu operáciu.

Pri spracovaní kryptomenových transakcií sme od klienta prijali podpísanú transakciu, overili jej formálnu správnosť a validitu kryptografického podpisu a následne sme ju odoslali do príslušnej blockchain siete cez externé API. Server pritom v žiadnom kroku nepracoval s privátnym kľúčom – ten zostával výhradne na strane klienta. Metódy komunikovali iba s repozitármi a nikdy nepristupovali k databáze priamo, čím sme zachovali striktné oddelenie vrstiev.

3.2.4 Controller vrstva

Controller vrstvu sme implementovali ako REST API rozhranie. Každý endpoint sme navrhli tak, aby najprv validoval vstupné údaje pomocou dátových modelov a validačných pravidiel. Následne sme overili prítomnosť a platnosť JWT tokenu vrátane kontroly podpisu a expirácie. Po úspešnom overení sme požiadavku delegovali na príslušnú servisnú metódu.

Odpovede sme vracali vo formáte JSON so štandardizovanou štruktúrou obsahujúcou výsledok operácie alebo informácie o chybe. Týmto spôsobom sme zabezpečili konzistentné rozhranie pre klientsku aplikáciu.

3.2.5 Komunikácia medzi klientom a serverom

Všetka komunikácia medzi klientom a serverom prebiehala výhradne prostredníctvom protokolu HTTPS, čím sme zabezpečili šifrovanie dát počas prenosu. Každá požiadavka klienta obsahovala JWT token v hlavičke, ktorý server overoval pred spracovaním akejkoľvek operácie. Server kontroloval podpis tokenu, jeho expiráciu a rolu používateľa, ktorá určovala, aké operácie mal daný účet povolené vykonávať.

3.2.6 Pozvánkový systém

Do registračného procesu sme zabudovali pozvánkový systém, ktorý obmedzoval vstup iba na oprávnených používateľov. Každý pozývaci kód bol uložený v databáze so záznamom o použití. Pri registrácii sme najprv overili existenciu a platnosť kódu, a až po úspešnom vytvorení účtu sme kód v rámci databázovej transakcie označili ako použitý. Tento mechanizmus zabráňoval opakovanému použitiu toho istého kódu aj pri súbežných požiadavkách.

3.2.7 Spracovanie chýb a výnimiek na serveri

Pri návrhu backendu sme kládli dôraz na spoľahlivé spracovanie chýb. Implementovali sme Global Exception Handler, ktorý zachytával všetky výnimky v rámci API. Tento mechanizmus zabezpečil, že klient nikdy nedostal nespracovanú alebo citlivú chybovú správu, ktorá by mohla odhaliť vnútornú štruktúru servera alebo databázy.

Každá výnimka bola zalogovaná prostredníctvom knižnice Serilog a zároveň API vracalo štruktúrovanú odpoveď vo forme JSON obsahujúcu len potrebné informácie pre klienta, napríklad typ, názov a popis chyby a identifikátor sledovania (Trace ID).

Mechanizmus bol navrhnutý tak, aby sa nové typy chýb mohli centrálnie spracovať bez potreby modifikácie jednotlivých endpointov. V kombinácii s kontrolou oprávnení a overovaním JWT tokenov tento systém zabezpečil vysokú odolnosť voči útokom a minimalizoval riziko nepredvídaných stavov pri komunikácii klient–server.

3.3 Implementácia autentifikácie a autorizácie

Pri implementácii autentifikácie a autorizácie sme sa zamerali na bezpečný a flexibilný systém správy používateľských účtov, tokenov a dvojfaktorovej autentifikácie.

3.3.1 Registrácia používateľa

Proces registrácie začína overením pozývacieho kódu, ktorý zabezpečoval, že do systému mohli vstupovať iba oprávnení používatelia. Server načítal kód z databázy, skontroloval, či nebol predtým použitý, a až po úspešnom vytvorení účtu ho v rámci databázovej transakcie označil ako použitý, čím sa zabránilo jeho opakovanému použitiu.

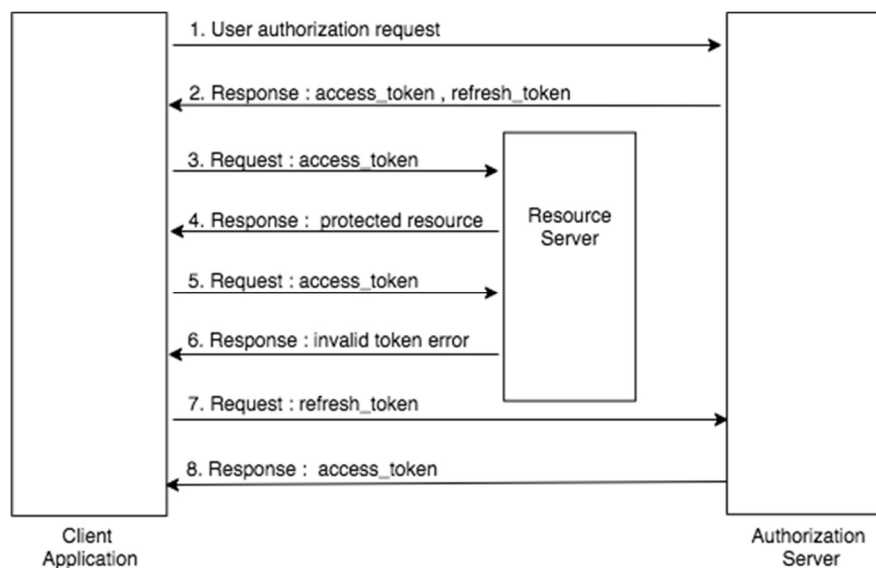
Používateľské heslá boli hashované pomocou algoritmu Argon2id, ktorý poskytuje vysokú odolnosť proti brute-force útokom aj rainbow table útokom. Pre každého používateľa sme vygenerovali unikátny náhodný salt, ktorý sme uložili do databázy spolu s výsledným hashom. Po úspešnom uložení záznamu server vrátil jedinečný identifikátor používateľa a desktopový klient zobrazil potvrdenie o úspešnej registrácii.

3.3.2 Autentifikácia používateľa

Pri prihlasovaní klient odoslal používateľské meno alebo e-mail spolu s heslom. Server najprv vyhľadal používateľa v databáze a overil, či účet nie je zablokovaný alebo odstránený. Heslo sme následne overili porovnaním s uloženým hashom Argon2id. Porovnanie sme vykonali v časovo konštantnom režime pomocou metódy FixedTimeEquals, čím sme eliminovali možnosť timing útoku, pri ktorom by útočník mohol na základe rýchlosti odpovede odvodiť čiastočnú zhodu hesla.

V prípade aktivovanej dvojfaktorovej autentifikácie server po overení hesla vyžadoval dodatočný jednorazový TOTP kód generovaný klientom. Ak kód chýbal alebo bol neplatný, server požiadavku odmietol a vrátil špecifickú chybovú odpoveď odlišnú od odpovede pri nesprávnom hesle.

Po úspešnej autentifikácii server vygeneroval krátkodobý JWT token s limitovanou platnosťou a dlhodobý refresh token viazaný na konkrétne zariadenie, ktoré boli následne odoslané klientovi.



Obrázok 2 Sekvenčný diagram autorizačného toku OAuth 2.0 (11)

3.3.3 Správa tokenov

3.3.3.1 Access Token

Access token sme implementovali ako krátkodobý JWT token podpísaný symetrickým kľúčom pomocou algoritmu HMAC SHA-256, určený na autorizáciu pri volaní API. Token obsahoval identifikátor používateľa, jeho rolu a unikátny identifikátor tokenu (jti). Po vypršaní platnosti access tokenu klient automaticky požiadaval o jeho obnovu pomocou refresh tokenu bez nutnosti opätovného zadávania hesla.

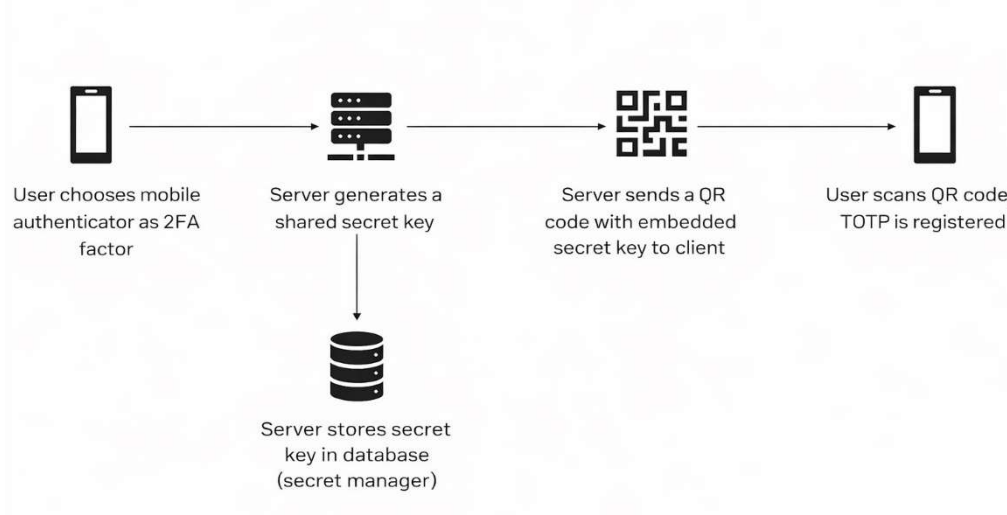
3.3.3.2 Refresh token

Refresh token sme generovali ako náhodný 32-bajtový reťazec, ktorý sme pred uložením do databázy hashovali algoritmom SHA-256. Každý refresh token bol priradený ku konkrétnemu zariadeniu – každému záznamu sme priradili informácie o zariadení vrátane IP adresy, operačného systému a hardvérového a sieťového identifikátora. Pri každej žiadosti o obnovu access tokenu sme porovnali odtlačok zariadenia s uloženým záznamom. Ak sa zariadenie nezhodovalo, server obnovu odmietol, čím sme zamedzili zneužitiu odcudzeného tokenu na iných zariadeniach.

Používateľ mohol kedykoľvek zrušiť prístup konkrétnemu zariadeniu alebo všetkým zariadeniam naraz, pričom táto operácia pri aktivovanej 2FA vyžadovala dodatočné overenie.

3.3.4 Dvojfaktorová autentifikácia (2FA)

Implementácia dvojstupňového overenia pozostáva z troch fáz: inicializácie, overenia a správy záložných kódov.



Obrázok 3 Diagram procesu inicializácie TOTP (12)

3.3.4.1 Inicializácia

Inicializácia prebehla tak, že server vygeneroval tajný kľúč, zakódoval ho do formátu Base32 a vložil do URI vo formáte *otpauth://totp*. Klient na základe tohto URI zobrazil QR kód, ktorý si používateľ naskenoval do svojej autentifikačnej aplikácie na správu jednorazových kódov.

3.3.4.2 Overenie

Používateľ zadal jednorazový TOTP kód a klient ho odoslal na server. Server dešifroval uložený tajný kľúč a overil platnosť kódu. Ak bol kód platný, overenie sa považovalo za úspešné.

3.3.4.3 Záložné kódy

Záložné kódy sme zaviedli pre prípad straty zariadenia s autentifikačnou aplikáciou. Server vygeneroval sériu jednorazových kódov, ktoré sme hashovali algoritmom SHA-256 a uložili do databázy. Každý kód bolo možné použiť práve raz a systém evidoval čas posledného generovania, čím sme zamedzili zneužitiu opakovaným generovaním nových súkromných kódov v krátkom čase.

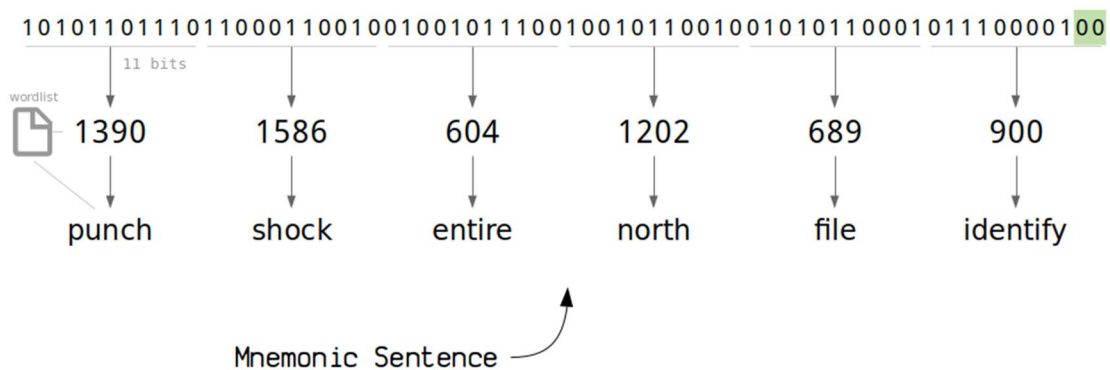
3.4 Vytváranie kryptomenovej peňaženky

Pri implementácii kryptomenovej peňaženky sme sa riadili zásadou, že citlivé údaje používateľa nikdy neopustia jeho lokálne prostredie. Celý proces tvorby peňaženky sme rozdelili do jasne definovaných krokov zabezpečujúcich bezpečné generovanie, ukladanie a správu privátnych kľúčov, seed fráz a transakcií. Tento prístup nám umožňoval kombinovať bezpečnosť, kontrolu nad dátami používateľa a efektívnu komunikáciu so serverom.

3.4.1 Generovanie mnemonic frázy a seedu

Prvým krokom pri vytváraní peňaženky bolo generovanie mnemonickej frázy. Využili sme štandard BIP39, ktorý umožňoval transformovať náhodnú entropiu do ľahko zapamätateľného slovného zoznamu. Pri tomto procese sme generovali entropiu s minimálnou dĺžkou 128 bitov, z ktorej vznikla 12-slovná mnemonicá fráza predstavujúca zálohovací ekvivalent celej peňaženky.

Na základe tejto frázy sme odvodili seed, z ktorého sa generovali privátne a verejné kľúče pre jednotlivé kryptomenové adresy. Celý tento proces prebiehal výhradne v pamäti klienta. Seed sme pred zápisom do lokálnej databázy LevelDB zašifrovali algoritmom AES-256, pričom šifrovací kľúč bol odvodený zo šesťmiestneho PIN-u používateľa. Rovnakým spôsobom sme šifrovali aj privátne kľúče.



Obrázok 4 Proces generovania mnemonickej frázy (BIP39) (13)

3.4.2 Inicializácia peňaženky a správa adresy

Po bezpečnom uložení seedu sme inicializovali samotnú peňaženku podľa štandardu BIP44, ktorý definuje derivačnú cestu pre každú podporovanú kryptomenu.

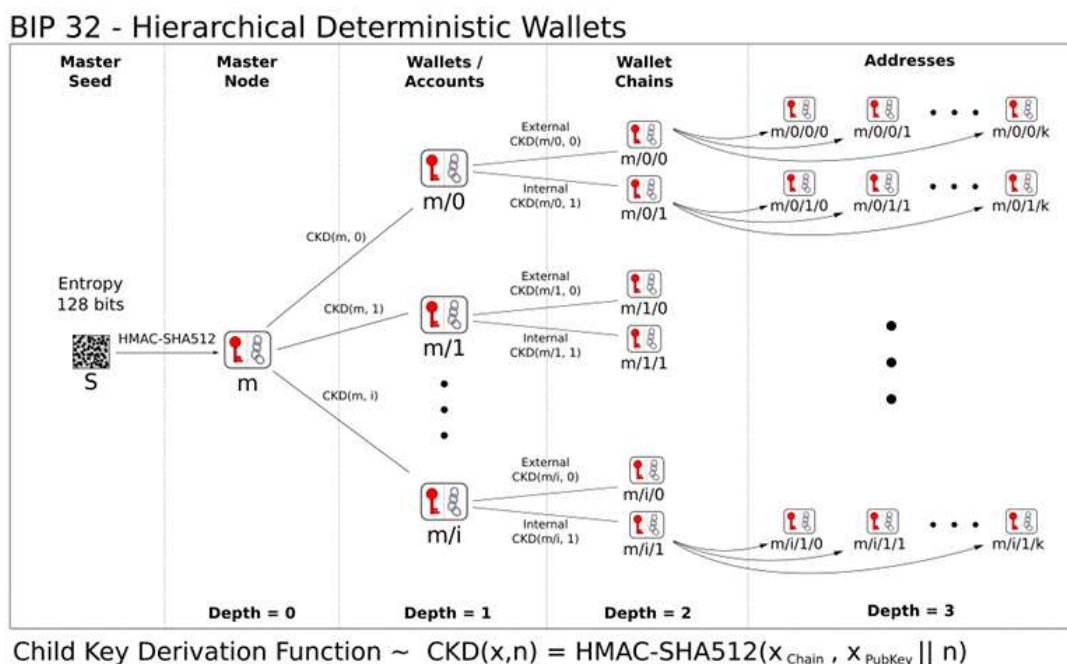
Z hlavného seedu sme odvodili účty a adresy pre jednotlivé meny, pričom každá adresa mala vlastný unikátny pár privátneho a verejného kľúča.

V rámci inicializácie peňaženky sme zabezpečili aj kontrolu integrity uložených dát. Pri každom spustení aplikácie klient najprv dešifroval seed v pamäti a testovou deriváciou verejnej adresy overil, že uložené šifrované dáta sú neporušené a správne. Ak testová derivácia zlyhala, aplikácia informovala používateľa o možnom poškodení dát pred akýmkoľvek ďalším krokom.

3.4.3 Import a export peňaženky

Import existujúcej peňaženky prebiehal zadaním mnemonickej frázy priamo v aplikácii. Klient overil integritu frázy, odvodil z nej seed a kľúče pre všetky podporované kryptomeny a peňaženku uložil do lokálnej databázy v šifrovanej podobe. Po úspešnom importe sa táto peňaženka stala aktívnou.

Export peňaženky bol obmedzený na mnemonicú frázu, čo umožňovalo používateľovi zálohovať prostriedky alebo presunúť peňaženku na iné zariadenie. Pri vytváraní novej peňaženky mal klient k dispozícii celú 12-slovnú frázu. Po jej akceptovaní a uložení klient vyžiadala potvrdenie, že si ju používateľ bezpečne uložil. Po skončení tohto kroku bola fráza v aplikácii nedostupná.



Obrázok 5 Hierarchicky deterministické peňaženky (BIP32) (14)

3.4.4 Aktívna peňaženka

Aby sme predišli neúmyselnému zaslaniu prostriedkov z nesprávnej peňaženky, zaviedli sme pravidlo jedinej aktívnej peňaženky. V danom okamihu mohla byť aktívna vždy iba jedna peňaženka – buď novovytvorená, alebo importovaná.

Prepnutie na inú peňaženku si vyžadovalo autentifikáciu používateľa, po ktorej klient odvodil seed novej peňaženky v pamäti a predchádzajúcu aktívnu peňaženku z pamäti úplne vymazal. Všetky operácie vrátane zobrazenia zostatku, histórie transakcií a odosielania sa vykonávajú výhradne pre aktuálne aktívnu peňaženku.

3.5 Správa peňaženky a integrácia rôznych kryptomien

Systém sme implementovali ako hierarchicky deterministickú (HD) peňaženku, čo nám umožnilo generovať adresy a kľúče pre viacero kryptomien z jediného seedu. Tento prístup poskytuje flexibilitu pri správe rôznych typov kryptomien a zároveň zaručuje vysokú úroveň bezpečnosti, keďže všetky privátne kľúče sú odvodené z jediného zdrojového seedu a nikdy neopúšťajú lokálne prostredie používateľa.

3.5.1 Synchronizácia zostatku a transakcií

Hoci privátne kľúče boli uchovávané výhradne lokálne, zostatok a história transakcií sa sprístupňovali cez backendový server. Klient odoslal na server verejnú adresu peňaženky, server požiadavku spracoval a prostredníctvom externého blockchain API príslušnej siete načítal relevantné dáta. Klientovi bola následne vrátená JSON odpoveď obsahujúca aktuálny zostatok, zoznam transakcií, ich stav a časové značky.

3.5.2 Realizácia transakcií

Odosielanie kryptomien sme rozdelili do troch základných krokov. V prvom kroku klient zostavil transakciu a lokálne ju podpísal privátnym kľúčom. Podpísaná transakcia obsahuje všetky údaje potrebné pre overenie v blockchain sieti vrátane vstupov, výstupov a sumy.

V druhom kroku klient odoslal podpísanú transakciu na server. Server vykonal kontrolu formátu transakcie, validitu kryptografického podpisu a následne transakciu odoslal do blockchain siete cez externé API. Server teda slúžil ako sprostredkovateľ, ktorý overoval a monitoroval transakcie, pričom nikdy neprichádzal do kontaktu s privátnym kľúčom.

Tretím krokom bola aktualizácia stavu transakcie a zostatku. Server prijal potvrdenie o zaevidovaní transakcie v sieti a poskytol klientovi spätnú väzbu. Klient zobrazil aktuálny stav transakcie používateľovi vrátane potvrdení a prípadných chýb pri jej spracovaní.

3.5.3 Podpora viacerých kryptomien

Pre každú podporovanú menu sme použili vlastnú derivačnú cestu podľa štandardu BIP44. Používateľ tak mohol spravovať napríklad Bitcoin aj Ethereum z jednej peňaženky bez potreby opätovného importu alebo generovania nového seedu. Derivácia kľúčov pre jednotlivé meny prebiehala v pamäti a klient spravoval tieto kľúče lokálne. Pri interakcii s konkrétnou kryptomenou klient selektoval správny derivovaný účet a príslušné adresy, čím umožňoval bezpečné odosielanie a príjem transakcií pre zvolenú menu.

4 VÝSLEDKY PRÁCE A DISKUSIA

4.1 Splnenie stanovených cieľov

Výsledkom projektu je plne funkčný informačný systém pozostávajúci z desktopovej aplikácie a backendového servera komunikujúcich prostredníctvom šifrovaného HTTPS spojenia. Systém podporuje registráciu pomocou pozvánkového kódu, viacúrovňové overovanie identity vrátane dvojfaktorovej autentifikácie a riadenie prístupových práv podľa používateľských rolí.

V oblasti správy peňaženiek aplikácia umožňuje vytváranie novej peňaženky aj import existujúcej pomocou mnemonickej frázy, šifrované lokálne ukladanie kryptografických údajov a bezpečné odosielanie transakcií. Všetky stanovené čiastkové ciele boli úspešne naplnené a overené počas vývoja aj testovania.

4.2 Testovanie a overenie funkčnosti

Testovanie prebiehalo priebežne počas celého vývoja a zameriavalo sa na tri hlavné oblasti: funkčnosť, bezpečnosť a stabilitu systému. V rámci autentifikačného procesu sme overili správnosť registrácie, prihlasovania, generovania a obnovy tokenov, fungovania dvojfaktorovej autentifikácie a korektného zneplatnenia prístupov pri odhlásení alebo zmene bezpečnostných nastavení.

Pri správe peňaženiek sme testovali generovanie aj import mnemonickej frázy, šifrované ukladanie dát, prístup pomocou PIN-u a obnovu stavu po reštarte aplikácie. Súčasťou testovania bolo aj podpisovanie a odosielanie transakcií, spracovanie chybových stavov a reakcia servera na súbežné požiadavky. Testovanie neodhalilo žiadne kritické bezpečnostné ani funkčné nedostatky.

4.3 Možnosti ďalšieho rozvoja

Do budúca je možné rozšíriť aplikáciu o podporu viacerých súčasne aktívnych peňaženiek a notifikácie o prichádzajúcich transakciách.

Z hľadiska bezpečnosti predstavuje perspektívne zlepšenie implementácia biometrickej autentifikácie a podpora hardvérových peňaženiek. Ďalším krokom môže byť rozšírenie podpory o ďalšie blockchain siete a tokenové štandardy, čím by sa zvýšila praktická využiteľnosť systému.

5 ZÁVER PRÁCE

Projekt BlockSense bol úspešne navrhnutý a implementovaný ako funkčná desktopová kryptopeňaženka. Všetky stanovené hlavné aj čiastkové ciele boli naplnené. Vytvorili sme bezpečný systém správy kryptomenových peňaženiek s plnohodnotným autentifikačným rámcom, intuitívnym používateľským rozhraním a dôkladnou ochranou citlivých kryptografických materiálov.

Kľúčovým prínosom práce je overenie praktickej realizovateľnosti princípu zero-knowledge v prostredí desktopovej aplikácie s centralizovaným backendom. Privátne kľúče a seed frázy v žiadnom okamihu neopustili lokálne prostredie používateľa, čím sa dosiahla vysoká úroveň bezpečnosti bez obmedzenia funkčnosti. Využitie novodobých štandardov zabezpečilo kompatibilitu s ostatnými peňaženkami a umožnilo správu viacerých kryptomien z jediného seedu.

V budúcnosti by sme chceli projekt rozšíriť o podporu ďalších kryptomien, integráciu hardvérových peňaženiek a biometrickej autentifikácie. Rovnako plánujeme rozšíriť systém o možnosť správy viacerých peňaženiek súčasne.

Z technologického hľadiska projekt priniesol praktické skúsenosti s vývojom viacvrstvových .NET aplikácií, implementáciou kryptografických mechanizmov, integráciou blockchain sietí a navrhovaním bezpečných autentifikačných systémov. Získané skúsenosti z tohto projektu budú cenným základom do odbornej praxe a pre ďalší rozvoj v oblasti softvérového inžinierstva a kryptografie.

ZHRNUTIE

Cieľom maturitnej práce bolo navrhnuť a implementovať desktopovú digitálnu peňaženku BlockSense určenú na bezpečnú správu kryptomenových prostriedkov. Aplikácia bola vyvinutá v prostredí .NET s využitím frameworku Avalonia UI pre klientsku časť a ASP.NET Core Web API pre serverovú časť. Dáta sa ukladajú v relačnej databáze MySQL na strane servera a v lokálnej databáze LevelDB na strane klienta.

Systém zahŕňa registráciu s pozvánkovým kódom, prihlasovanie s hashovaním hesiel, správu JWT a refresh tokenov viazaných na zariadenie a dvojfaktorovú autentifikáciu prostredníctvom TOTP. Správa peňaženiek je realizovaná podľa štandardov BIP39 a BIP44 s dôsledným uplatnením princípu zero-knowledge – privátne kľúče ani seed frázy nikdy neopustili lokálne prostredie používateľa. Šifrovanie citlivých dát zabezpečuje ich ochranu aj pri fyzickom prístupe k zariadeniu. Výsledkom je funkčná, bezpečná a používateľsky prívetivá aplikácia, ktorá naplňa všetky ciele stanovené v zadaní práce.

ZOZNAM POUŽITEJ LITERATÚRY

1. **Microsoft.** NET introduction. *Microsoft Learn*. [Online] [Dátum: 02. Február 2026.] <https://learn.microsoft.com/en-us/dotnet/core/introduction>.
2. **Wikipedia contributors.** ASP.NET Core. *Wikipedia*. [Online] [Dátum: 02. Február 2026.] https://en.wikipedia.org/wiki/ASP.NET_Core.
3. **Avalonia.** Cross-platform .NET UI framework. *Avalonia Documentation*. [Online] [Dátum: 02. Február 2026.] <https://docs.avaloniaui.net/docs/overview/what-is-avalonia>.
4. **Oracle Corporation.** MySQL 8.4 Reference Manual: What is MySQL? *MySQL Reference Manual*. [Online] [Dátum: 02. Február 2026.] <https://dev.mysql.com/doc/refman/8.4/en/what-is-mysql.html>.
5. **Wikipedia contributors.** Application programming interface. *Wikipedia*. [Online] [Dátum: 02. Február 2026.] https://sk.wikipedia.org/wiki/Application_programming_interface.
6. **OAuth.** OAuth 2.0. *OAuth.net*. [Online] [Dátum: 02. Február 2026.] <https://oauth.net/2/>.
7. **Wikipedia contributors.** Git. *Wikipedia*. [Online] [Dátum: 02. Február 2026.] <https://en.wikipedia.org/wiki/Git>.
8. **Bouncy Castle Inc.** About Bouncy Castle. *Bouncy Castle*. [Online] [Dátum: 02. Február 2026.] <https://www.bouncycastle.org/about/>.
9. **IBM Corporation.** What is blockchain? *IBM Think*. [Online] [Dátum: 02. Február 2026.] <https://www.ibm.com/think/topics/blockchain>.
10. **Pro Code Guide.** Repository Pattern in ASP.NET Core with Adapter Pattern. *Pro Code Guide*. [Online] [Dátum: 26. Február 2026.] <https://procodeguide.com/programming/repository-pattern-in-aspnet-core/>.
11. **WSO2 Identity Server Documentation.** Refresh Token Grant Type. *Identity Server Documentation*. [Online] [Dátum: 26. Február 2026.] <https://is.docs.wso2.com/en/6.1.0/references/concepts/authorization/refresh-token-grant/>.

12. **Kittipat.Po.** Implementing Two-Factor Authentication (2FA) with TOTP in Golang. *Medium*. [Online] [Dátum: 26. Febrúar 2026.] https://medium.com/@kittipat_1413/implementing-two-factor-authentication-2fa-with-totp-in-golang-1a3fd0dd7662.

13. **Walker, Greg.** Mnemonic Seed . *Learn Me A Bitcoin*. [Online] [Dátum: 26. Febrúar 2026.] <https://learnmeabitcoin.com/technical/keys/hd-wallets/mnemonic-seed/>.

14. **Wuille, Pieter.** BIP 32: Hierarchical Deterministic Wallets. *Bitcoin Improvement Proposal*. [Online] [Dátum: 26. Febrúar 2026.] <https://bips.dev/32/>.

ZOZNAM PRÍLOH

PRÍLOHA A POUŽÍVATEĽSKÁ PRÍRUČKA	32
PRÍLOHA B SCHÉMA DATABÁZY	39
PRÍLOHA C ZDROJOVÝ KÓD DATABÁZOVÉHO TRIGGERA	40
PRÍLOHA D LOKÁLNE ULOŽENÝ OBNOVOVACÍ TOKEN.....	41
PRÍLOHA E LOKÁLNE APLIKAČNÉ LOG SÚBORY	42
PRÍLOHA F LOKÁLNA DATABÁZA PEŇAŽENKY	43
PRÍLOHA G OBRAZOVKA REGISTRÁCIE POUŽÍVATEĽA	44
PRÍLOHA H OBRAZOVKA PRIHLÁSENIA POUŽÍVATEĽA	45
PRÍLOHA I UVÍTACIA OBRAZOVKA APLIKÁCIE	46
PRÍLOHA J KONFIGURÁCIA DVOJFAKTOROVEJ AUTENTIFIKÁCIE .	47
PRÍLOHA K SPRÁVA DVOJFAKTOROVEJ AUTENTIFIKÁCIE.....	48
PRÍLOHA L GENEROVANIE TOTP KÓDU	49
PRÍLOHA M DIALÓG OVERENIA TOTP KÓDU.....	50
PRÍLOHA N SPRÁVA AKTÍVNYCH ZARIADENÍ.....	51
PRÍLOHA O ZOZNAM AKTIVITY POUŽÍVATEĽA	52
PRÍLOHA P SPRÁVA POZVÁNOK POUŽÍVATEĽA.....	53
PRÍLOHA Q POTVRDENIE PIN KÓDU PEŇAŽENKY	54
PRÍLOHA R PROJEKTOVÉ SÚBORY	55
PRÍLOHA S FINÁLNY VÝSTUP PROJEKTU	56
PRÍLOHA T PREZENTAČNÉ VIDEO	57

BLOCKSENSE

Digitálna kryptomenová peňaženka

POUŽÍVATEĽSKÁ PRÍRUČKA

1 Úvod

Táto príručka popisuje inštaláciu, konfiguráciu a každodennú obsluhu desktopovej aplikácie BlockSense – kryptomenovej peňaženky vyvinutej v rámci maturitnej práce na Strednej priemyselnej škole Jozefa Murgaša v Banskej Bystrici.

BlockSense umožňuje bezpečne spravovať kryptomenové prostriedky bez toho, aby privátne kľúče alebo seed fráza opustili lokálne prostredie zariadenia. Dokument je určený koncovým používateľom a predpokladá základnú znalosť práce s počítačom.

Pre správne pochopenie príručky odporúčame prečítať kapitoly v poradí, v akom sú usporiadané, predovšetkým ak aplikáciu používate po prvýkrát.

2 Systémové požiadavky

2.1 Hardvérové požiadavky

Procesor	Minimálne dvojjadrový CPU s frekvenciou 2 GHz alebo vyššou
Operačná pamäť	Minimálne 2 GB RAM; odporúčané 4 GB alebo viac
Úložisko	Minimálne 100 MB voľného miesta pre metadata a lokálnu databázu
Sieťové pripojenie	Stabilné internetové pripojenie (potrebné pre synchronizáciu zostatku a odosielanie transakcií)

2.2 Softvérové požiadavky

Windows	Windows 7 (x64) alebo novší; odporúčané Windows 10 / 11
macOS	macOS 10.13 High Sierra alebo novší; odporúčané 10.15 Catalina+
.NET Runtime	.NET 8 Runtime alebo novší (dostupný na stránke microsoft.com/net)

3 Registrácia používateľského účtu

BlockSense využíva uzatvorený registračný systém chránený pozývacím kódom. Registrácia je prístupná iba používateľom, ktorí disponujú platným pozývacím kódom vydaným správcom systému.

3.1 Postup registrácie

1. Po spustení aplikácie kliknite na tlačidlo Registrovať sa (*Register*) na úvodnej obrazovke.
2. Zadať požadované registračné údaje:
 - Používateľské meno (jedinečné v rámci systému, minimálne 4 znaky)
 - E-mailovú adresu
 - Heslo (minimálne 8 znakov, kombinácia veľkých a malých písmen, číslíc a špeciálnych znakov)
 - Platný pozývací kód
3. Potvrďte registráciu kliknutím na tlačidlo Vytvoriť účet (*Register*).
4. Po úspešnom vytvorení účtu budete automaticky presmerovaný na obrazovku prihlasovania.

 **Poznámka:** Každý pozývací kód je jednorazový a po jeho použití sa stáva neplatným. V prípade problémov s registráciou kontaktujte správcu systému.

3.2 Bezpečnostné odporúčania pre heslo

- Heslo musí mať minimálne 8 znakov.
- Odporúčame použiť kombináciu veľkých písmen, malých písmen, číslíc a špeciálnych znakov (napr. @, #, !).
- Nepoužívajte heslo, ktoré ste využívali na iných platformách alebo službách.
- Heslo si bezpečne uložte – aplikácia nedisponuje funkciou automatického obnovenia hesla cez e-mail.
- Zvážte použitie správcu hesiel (password manager) pre bezpečné ukladanie prihlasovacích údajov.

4 Prihlásenie do aplikácie

4.1 Štandardné prihlásenie

1. Po spustení aplikácie kliknite na tlačidlo Overiť (*Authenticate*) na úvodnej obrazovke.
2. Zadať požadované prihlasovacie údaje:
 - Používateľské meno alebo e-mailovú adresu
 - Heslo
3. Kliknite na tlačidlo Overiť (*Authenticate*).
4. Po úspešnom prihlásení budete presmerovaný na hlavný panel aplikácie (*Dashboard*).

4.2 Prihlásenie s aktívnou dvojfaktorovou autentifikáciou

1. Vyplňte používateľské meno a heslo podľa krokov v sekcii 4.1.
2. Aplikácia zobrazí výzvu na zadanie jednorazového kódu (TOTP).
3. Otvorte vašu autentifikačnú aplikáciu (napr. Google Authenticator, Authy alebo Microsoft Authenticator) a skopírujte aktuálny 6-ciferný kód.
4. Zadaťte kód do príslušného poľa a kliknite na tlačidlo Overiť (*Verify*).

⚠ Poznámka: Jednorazový TOTP kód je platný iba po dobu 30 sekúnd. Ak uplynie čas jeho platnosti, vyčkajte na vygenerovanie nového kódu a skúste znova.

4.3 Prihlásenie pomocou záložného kódu

Ak nemáte prístup k zariadeniu s autentifikačnou aplikáciou, môžete sa prihlásiť pomocou záložného kódu:

1. Na obrazovke TOTP overovania kliknite na tlačidlo Overiť pomocou záložného kódu (*Verify using backup code*) v ľavom dolnom rohu.
2. Zadaťte jeden z vopred vygenerovaných záložných kódov.
3. Potvrďte kliknutím na tlačidlo Overiť (*Verify*).

⚠ Poznámka: Každý záložný kód možno použiť iba raz. Po použití je zo systému automaticky odstránený. Pokiaľ ste použili všetky záložné kódy, vygenerujte novú sadu v nastaveniach zabezpečenia.

5 Nastavenia bezpečnosti

5.1 Aktivácia dvojfaktorovej autentifikácie

Dvojfaktorová autentifikácia výrazne zvyšuje bezpečnosť vášho účtu. Po jej aktivácii je pri každom prihlásení okrem hesla vyžadovaný jednorazový kód z autentifikačnej aplikácie.

1. Po prihlásení kliknite na tlačidlo Používateľský Profil (*User Profile*).
2. Prejdite do sekcie Bezpečnostné Nastavenia (*Security Settings*) a kliknite na Aktivovať 2FA (*Enable 2FA*).
3. Nainštalujte autentifikačnú aplikáciu do vášho mobilného zariadenia (napr. Google Authenticator, Authy).
4. Vložte bezpečnostný kľúč alebo naskenujte zobrazený QR kód pomocou autentifikačnej aplikácie.
5. Zadaťte 6-ciferný kód vygenerovaný aplikáciou pre overenie správnosti nastavenia.

5.2 Správa záložných kódov

Záložné kódy slúžia ako núdzový prístup k účtu v prípade, že nemáte prístup k zariadeniu s autentifikačnou aplikáciou. Každý kód je jednorazový.

Novú sadu záložných kódov môžete vygenerovať v sekcii Bezpečnostné nastavenia (*Security Settings*) kliknutím na tlačidlo Generovať kódy (*Generate Codes*). Vygenerovaním novej sady sa všetky predchádzajúce kódy zneplatnia. Túto operáciu je možné vykonať každú 1 hodinu, preto si kódy uložte kliknutím na tlačidlo Stiahnuť .txt súbor (*Download *.txt file*).

5.3 Správa zariadení a relácií

V sekcii Zariadenia (*Active Devices*) vidíte zoznam všetkých zariadení, na ktorých je váš účet aktuálne prihlásený. Pre každé zariadenie sú uvedené informácie o IP adrese zariadenia, čase prihlásenia a čase vypršania relácie.

Prístup konkrétnemu zariadeniu môžete zrušiť kliknutím na tlačidlo Odhlásiť zariadenie (*Revoke*). Táto operácia pri aktívnej dvojfaktorovej autentifikácii vyžaduje overenie TOTP kódom. Pre odhlásenie všetkých zariadení naraz použite možnosť Odhlásiť všetky dostupné zariadenia (*Log Out All Known Devices*).

⚠ Poznámka: Ak spozorujete zariadenie alebo IP adresu, ktorú nepoznáte, okamžite zrušte príslušný prístup alebo kontaktujte správcu systému.

6 Odhlásenie z aplikácie

Pre bezpečné odhlásenie prejdite do sekcie Zariadenia (*Active Devices*). Vaše aktuálne zariadenie bude označené ako Toto Zariadenie (*This Device*). Kliknutím na tlačidlo Odhlásiť sa (*Sign out*) sa aktuálna relácia zneplatní a budete presmerovaní na prihlasovaciu obrazovku.

⚠ Poznámka: Privátne kľúče a seed fráza sú po odhlásení okamžite odstránené z operačnej pamäte a disku.

✓ **Odporúčanie:** Na zdieľaných zariadeniach vždy využite funkciu odhlásenia. Nikdy nezatvárajte aplikáciu bez odhlásenia.

7 Správa pozvánok

V sekcii Používateľský Profil (*User Profile*) je možné otvoriť nové okno so správou pozvánok kliknutím na tlačidlo Správca Pozvánok (*Invitation Manager*).

7.1 Prehľad a správa existujúcich kódov

V okne Invitation Manager sa zobrazí zoznam všetkých vygenerovaných pozvánok. Pre každú pozvánku sú zobrazené nasledovné informácie: dátum vytvorenia, dátum expirácie, samotný kód, používateľ, ktorý kód použil (ak bol použitý) a stav. Pre zobrazenie kódu, kliknite na tlačidlo Kliknutím sem Rozbalíte (*Click here to Expand*).

⚠ Poznámka: Generovanie a pridelovanie pozvánok je operácia dostupná výhradne pre administrátorov systému. Bežný používateľ k tejto funkcii v aplikácii nemá prístup.

8 Používateľská aktivita

Sekcia Nedávna Aktivita (*Recent Activity*) zaznamenáva bezpečnostne relevantné udalosti viazané na váš účet, ako napríklad pokusy o prihlásenie, zmeny bezpečnostných nastavení alebo odhlásenie zariadenia. Tieto záznamy slúžia na monitorovanie aktivity a detekciu podozrivých akcií.

8.1 Zobrazenie logov aktivity

1. Po prihlásení kliknite na tlačidlo Používateľský Profil (*User Profile*).
2. Prejdite do sekcie Nedávna Aktivita (*Recent Activity*) a kliknite na tlačidlo Zobrazíť Celý Záznam Aktivít (*View Full Activity Log*).
3. Zobrazí sa zoznam posledných aktivít zoradených od najnovšej. Pre každú akciu sú zobrazené: dátum vykonania, samotná akcia a typ udalosti.

8.2 Typy zaznamenávaných udalostí

Na základe zaznamenanej aktivity systém rozlišuje tri hlavné typy udalostí:

- **Používateľská aktivita (User)** – akcie, ktoré vykonáte vy, napríklad prihlásenie, odhlásenie, zmena hesla alebo úprava profilu.
- **Systémová aktivita (System)** – automatické procesy systému, napr. aktualizácie nastavení alebo kontrola bezpečnosti.
- **Plánované úlohy (Cron)** – pravidelné alebo naplánované úlohy, napr. automatické odhlásenie neaktívnych používateľov.

⚠ Poznámka: Ak spozorujete aktivitu, ktorú ste nevykonali (napríklad prihlásenie z neznámej IP adresy), okamžite zrušte prístupy všetkých zariadení a kontaktujte správcu systému.

9 Správa kryptomenovej peňaženky

9.1 Vytvorenie novej peňaženky

Peňaženka sa vytvára iba raz a jej zálohou je 12-slovná mnemonická fráza. Tento proces prebieha výhradne lokálne na vašom zariadení.

1. V sekcii Moja Peňaženka (*My Wallet*) kliknite na Vytvoriť Novú Peňaženku (*Create New Wallet*).
2. Aplikácia vygeneruje unikátnu 12-slovnú mnemonickú frázu. Táto fráza je zobrazená iba raz.
3. Frázu si bezpečne zapíšte v presnom poradí na papier a uložte ho na bezpečné miesto. Nikdy ju neuchovávajte v digitálnej podobe na rovnakom zariadení.
4. Nastavte 6-ciferný PIN kód, ktorý bude chrániť prístup k peňaženke v rámci aplikácie.
5. Potvrďte PIN jeho opätovným zadaním.
6. Kliknite na tlačidlo Potvrdiť (*Confirm*). Peňaženka je teraz aktívna a pripravená na použitie.

⚠ Poznámka: Mnemonická fráza je jediným prostriedkom na obnovenie peňaženky. Ak ju stratíte, vaše prostriedky budú nenávratne nedostupné. Aplikácia nedisponuje žiadnym spôsobom obnovenia bez frázy.

9.2 Import existujúcej peňaženky

Ak disponujete existujúcou peňaženkou (napr. z inej aplikácie alebo zariadenia), môžete ju importovať pomocou mnemonickkej frázy.

1. V sekcii Moja Peňaženka (*My Wallet*) kliknite na tlačidlo Importovať existujúcu peňaženku (*Import Existing Wallet*).
2. Zadaťte vašu 12-slovnú mnemonickú frázu v správnom poradí.
3. Nastavte 6-ciferný PIN kód, ktorý bude chrániť prístup k peňaženke v rámci aplikácie.
4. Potvrďte PIN jeho opätovným zadaním.
5. Kliknite na tlačidlo Potvrdiť (*Confirm*). Aplikácia odvodí privátne kľúče a adresár kryptomien.
6. Peňaženka je teraz aktívna a pripravená na použitie.

9.3 Zobrazenie zostatku a histórie transakcií

Po prihlásení a odomknutí peňaženky PIN kódom zobrazí hlavný panel aktuálny zostatok pre každú podporovanú kryptomenu. Zostatok je synchronizovaný prostredníctvom servera s externou blockchain sieťou.

História transakcií zobrazuje odosielateľa, príjemcu, sumu, stav transakcie a časovú pečiatku. Zoznam je možné aktualizovať prostredníctvom tlačidla Obnoviť (*Refresh*).

10 Realizácia transakcií

10.1 Prijatie kryptomeny

Na prijatie kryptomeny nie je potrebné vykonať žiadnu transakciu. Stačí poskytnúť odosielateľovi vašu verejnú adresu peňaženky.

1. V hlavnom paneli vyberte požadovanú kryptomenu.
2. Aplikácia zobrazí vašu verejnú adresu vo forme textu.
3. Skopírujte adresu kliknutím na ikonu kopírovania umiestnenú napravo od textu.

10.2 Odoslanie kryptomeny

1. V hlavnom paneli vyberte kryptomenu, ktorú chcete odoslať.
2. Kliknite na tlačidlo Odoslať (*Send*).
3. Zadaťte adresu príjemcu. Adresu môžete zadať manuálne alebo vložiť zo schránky.
4. Zadaťte sumu, ktorú chcete odoslať a skontrolujte zobrazené poplatky za transakciu (*Transaction fee*).
5. Kliknite na tlačidlo Pokračovať (*Continue*) a zadaťte váš 6-ciferný PIN kód na potvrdenie transakcie.
6. Aplikácia lokálne podpíše transakciu a odošle ju na server, ktorý ju zaeviduje v blockchain sieti.
7. Po odoslaní sa transakcia objaví v histórii so stavom Čakajúca (*Pending*). Po potvrdení sieťou sa stav zmení na Potvrdená (*Confirmed*).

⚠️ Poznámka: *Pred potvrdením transakcie dôkladne skontrolujte adresu príjemcu. Odoslanie kryptomeny na nesprávnu adresu je nevratné. Aplikácia ani server nemôžu transakciu odvolať po jej zaevidovaní v blockchain sieti.*

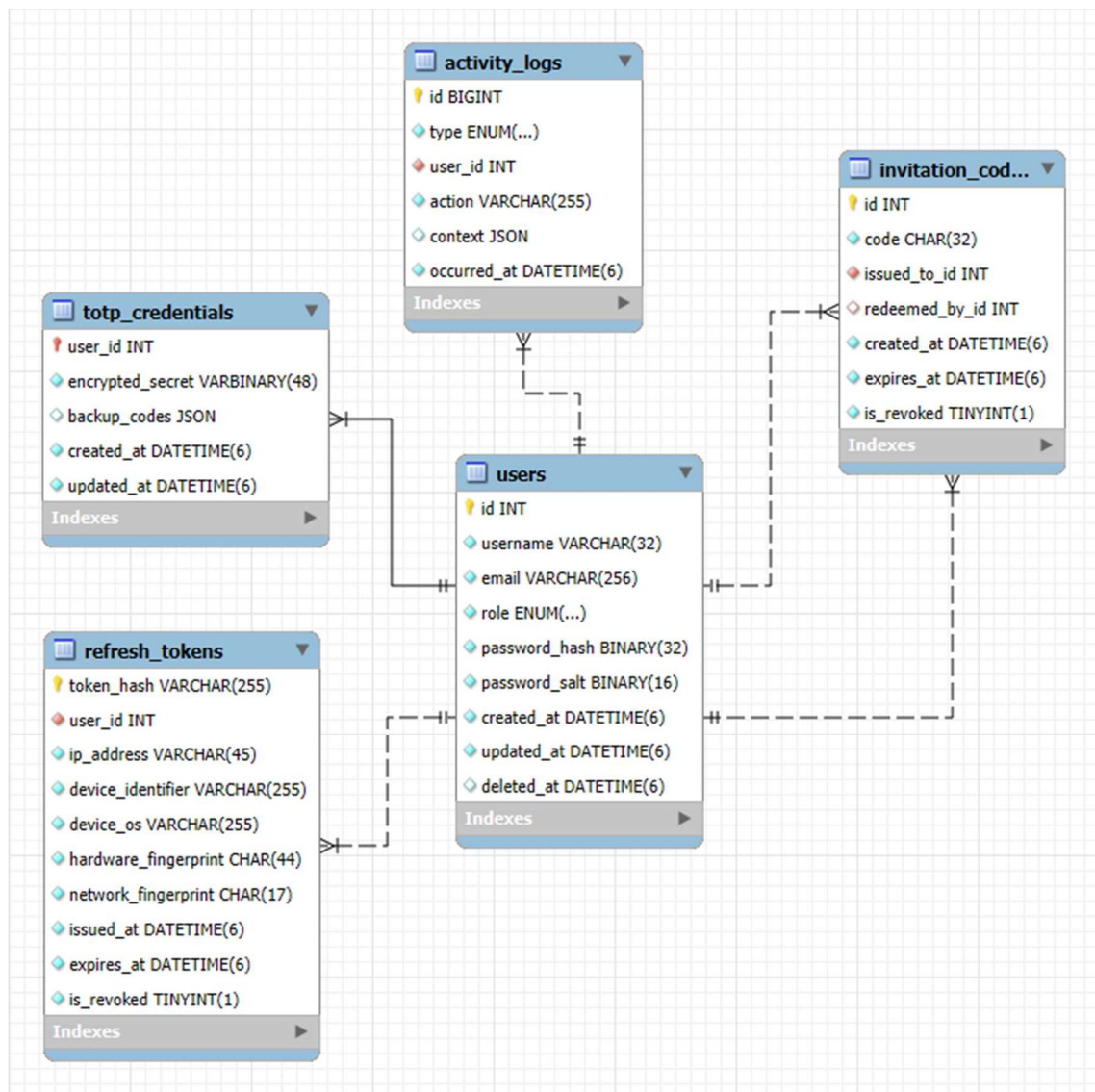
✓ Odporúčanie: *Pre vysoké sumy odporúčame najskôr odoslať testovaciu transakciu malej hodnoty a overiť, že adresa príjemcu je správna.*

11 Riešenie bežných problémov

11.1 Prehľad najčastejších problémov

Problém	Možné riešenie
Aplikácia sa nespustí	Skontrolujte, či máte nainštalovaný .NET Runtime 8 alebo novší. Skúste spustiť aplikáciu s oprávneniami správcu.
Nepodarilo sa prihlásenie	Skontrolujte správnosť používateľského mena a hesla. Overte, či je klávesnica v správnom jazyku (CapsLock, rozloženie).
Neplatný TOTP kód	Uistite sa, že čas na vašom zariadení je synchronizovaný (NTP). TOTP kódy sú závislé na presnom čase.
Zostatok sa nezobrazuje	Skontrolujte internetové pripojenie. Kliknite na ikonu obnovenia vedľa zostatku. Ak problém pretrváva, skontrolujte, či je server BlockSense dostupný.
Transakcia zostáva v stave Čakajúca	Stav Čakajúca je normálny počas spracovania blockchain sieťou. Čas potvrdenia závisí od zvoleného poplatku a aktuálneho zaťaženia siete.
Zabudnutý PIN peňaženky	Peňaženku nie je možné obnoviť bez PIN kódu. Importujte peňaženku znova pomocou pôvodnej 12-slovnej frázy a nastavte nový PIN.
Aplikácia nereaguje	Ukončite aplikáciu pomocou správcu úloh (Ctrl+Alt+Del na Windows) a spustite ju znova. Ak problém pretrváva, kontaktujte správcu systému.

PRÍLOHA B SCHÉMA DATABÁZY



PRÍLOHA C ZDROJOVÝ KÓD DATABÁZOVÉHO TRIGGERA

```
1 CREATE DEFINER='root'@'localhost' TRIGGER `activity_logs_AFTER_INSERT` AFTER INSERT ON `activity_logs` FOR EACH ROW BEGIN
2   -- Update the corresponding user's updated_at to the log timestamp
3   UPDATE `users`
4     SET `updated_at` = NEW.occurred_at
5     WHERE `id` = NEW.user_id;
6 END
```


PRÍLOHA D LOKÁLNE ULOŽENÝ OBNOVOVACÍ TOKEN

c \ [none_] 723 222 764 k of 975 984 996 k free \ ..				
c:\Users\d3str\AppData\Roaming\BlockSense\auth*.*				
↓ Name	Ext	Size	Date	Attr
↑ [..]		<DIR>	24.02.2026 20:32	----
refresh_token	bin	326	24.02.2026 20:32	-a--

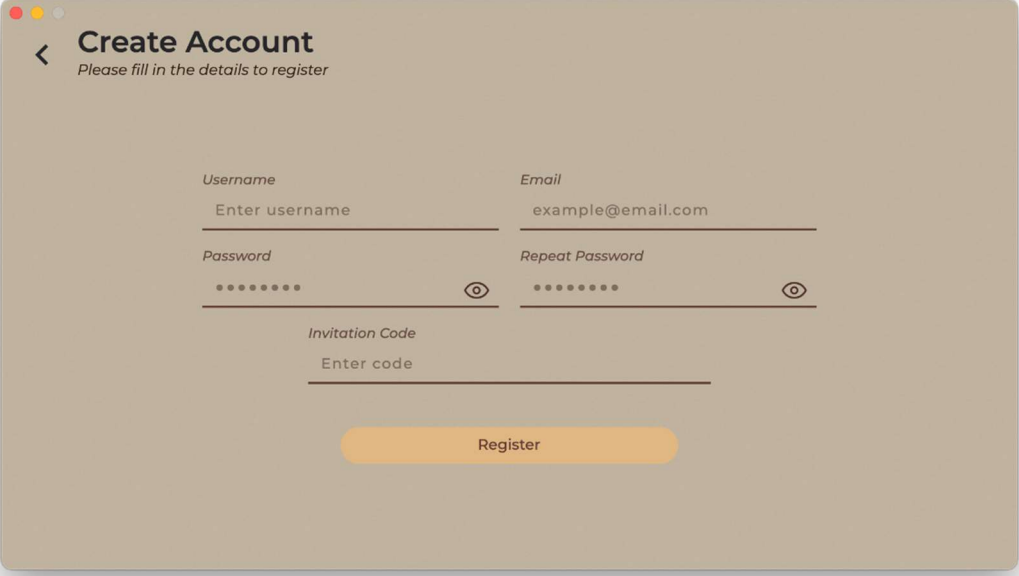
PRÍLOHA E LOKÁLNE APLIKAČNÉ LOG SÚBORY

c: \ [none_] 723 223 020 k of 975 984 996 k free				
c:\Users\d3str\AppData\Roaming\BlockSense\logs*.*				
↓ Name	Ext	Size	Date	Attr
↑ [..]		<DIR>	26.02.2026 18:33	----
blocksense20260226	log	29 135	26.02.2026 19:41	-a--
blocksense20260225	log	74 157	25.02.2026 19:53	-a--
blocksense20260224	log	115 313	24.02.2026 21:12	-a--
blocksense20260223	log	4 321	23.02.2026 22:49	-a--
blocksense20260219	log	78	19.02.2026 21:34	-a--
blocksense20260217	log	84 464	17.02.2026 21:43	-a--
blocksense20260216	log	3 078	16.02.2026 20:43	-a--
blocksense20260212	log	7 032	12.02.2026 17:44	-a--
blocksense20260211	log	6 324	11.02.2026 21:04	-a--
blocksense20260207	log	12 406	07.02.2026 20:22	-a--
blocksense20260203	log	300	03.02.2026 19:52	-a--

PRÍLOHA F LOKÁLNA DATABÁZA PEŇAŽENKY

c: \ [none_] 722 912 936 k of 975 984 996 k free				
c:\Users\d3str\AppData\Roaming\BlockSense\wallet*.*				
↑ Name	Ext	Size	Date	Attr
[.]		<DIR>	26.02.2026 23:56	----
000004	log	28	26.02.2026 23:56	-a--
CURRENT		16	26.02.2026 23:56	-a--
IDENTITY		36	26.02.2026 23:56	-a--
LOCK		0	26.02.2026 23:56	-a--
LOG		31 940	26.02.2026 23:56	-a--
MANIFEST-000005		116	26.02.2026 23:56	-a--
OPTIONS-000007		7 864	26.02.2026 23:56	-a--

PRÍLOHA G OBRAZOVKA REGISTRÁCIE POUŽÍVATEĽA



The image shows a 'Create Account' registration form within a window. The window has a title bar with three colored buttons (red, yellow, grey) on the top left. The form has a light beige background. At the top left of the form is a back arrow icon and the title 'Create Account' in bold, followed by the subtitle 'Please fill in the details to register'. The form contains four input fields: 'Username' with the placeholder 'Enter username', 'Email' with the placeholder 'example@email.com', 'Password' with a masked password '.....' and an eye icon to toggle visibility, and 'Repeat Password' with a masked password '.....' and an eye icon. Below these is an 'Invitation Code' field with the placeholder 'Enter code'. At the bottom center is an orange rounded button labeled 'Register'.

Create Account
Please fill in the details to register

Username
Enter username

Email
example@email.com

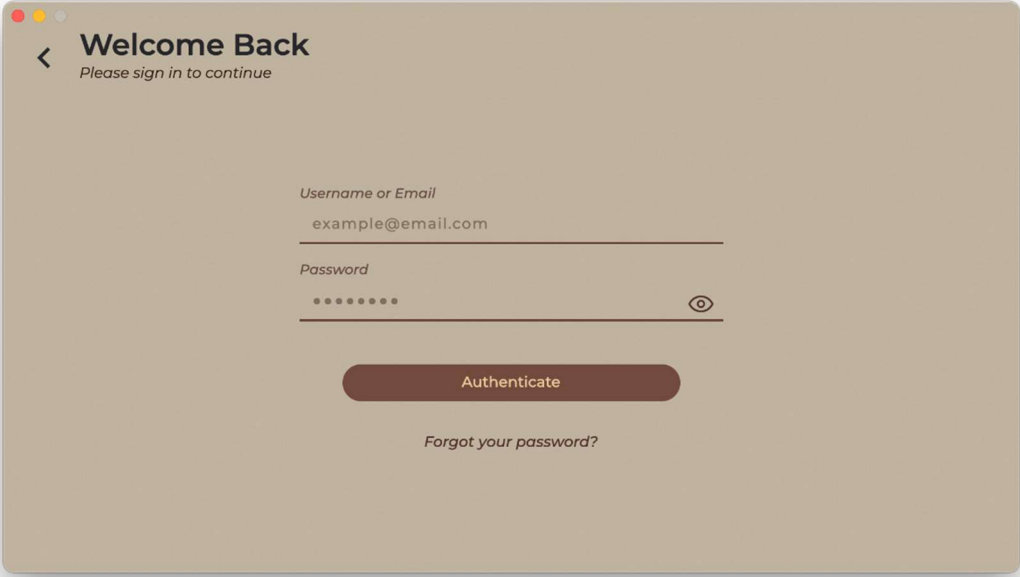
Password
..... 

Repeat Password
..... 

Invitation Code
Enter code

Register

PRÍLOHA H OBRAZOVKA PRIHLÁSENIA POUŽÍVATEĽA



A user login screen mockup with a light beige background. At the top left, there are three colored window control buttons (red, yellow, grey) and a back arrow icon. The title 'Welcome Back' is in bold, with the subtitle 'Please sign in to continue' below it. The form contains two input fields: 'Username or Email' with the placeholder 'example@email.com' and 'Password' with masked characters '••••••••'. A toggle eye icon is to the right of the password field. Below the fields is a dark brown 'Authenticate' button. At the bottom, there is a link 'Forgot your password?'.

< **Welcome Back**
Please sign in to continue

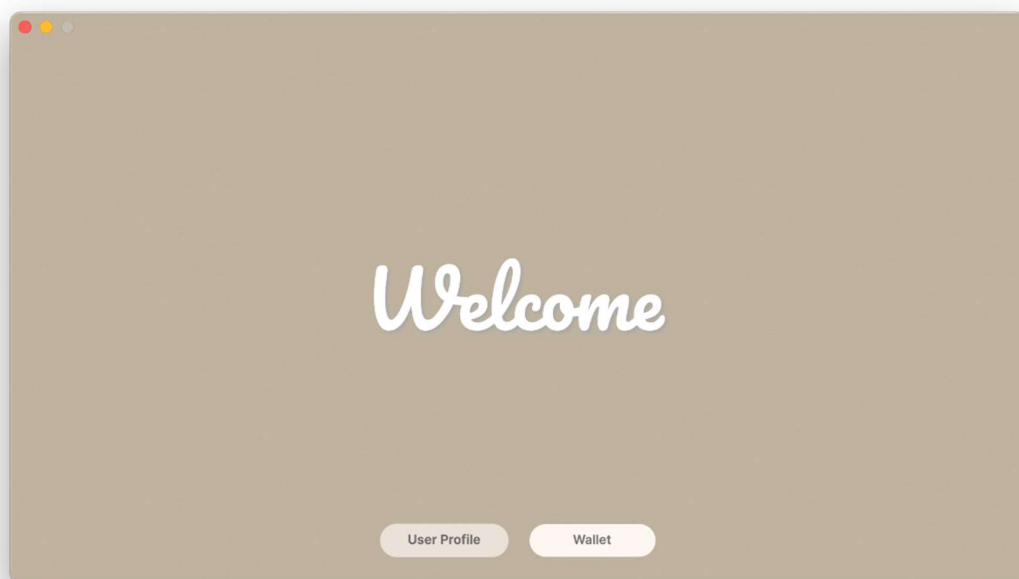
Username or Email
example@email.com

Password
•••••••• 

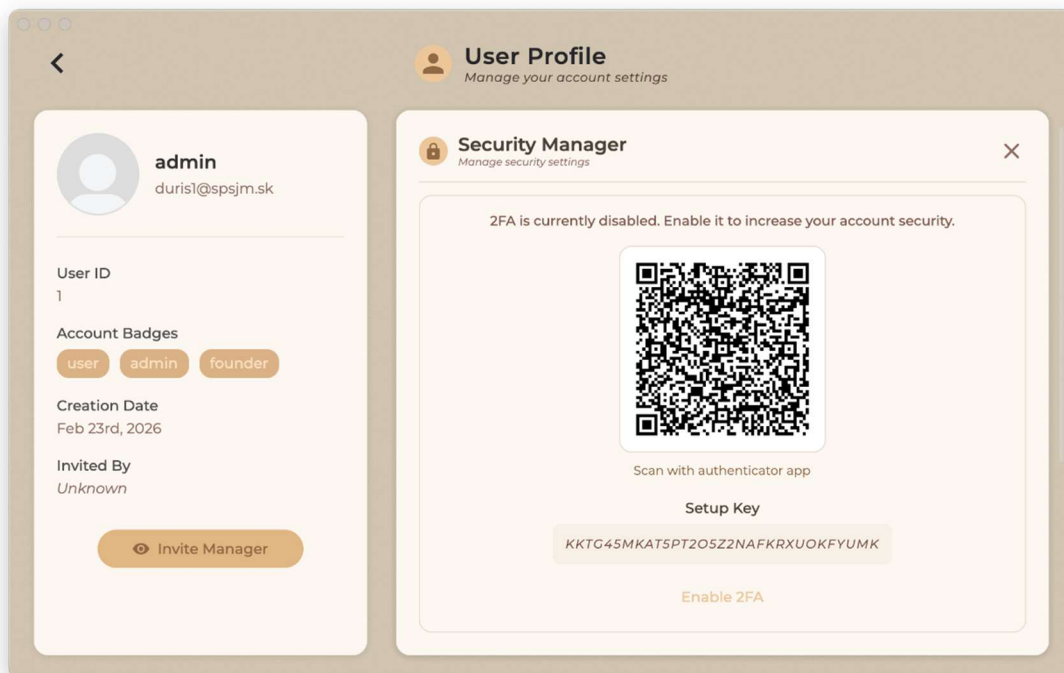
Authenticate

[Forgot your password?](#)

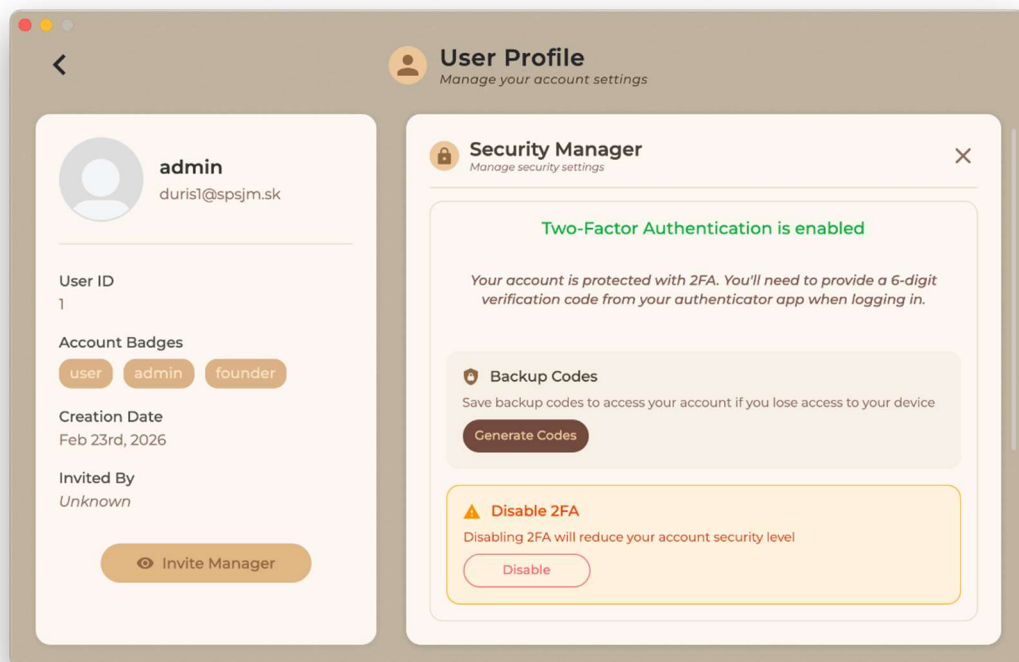
PRÍLOHA I UVÍTACIA OBRAZOVKA APLIKÁCIE



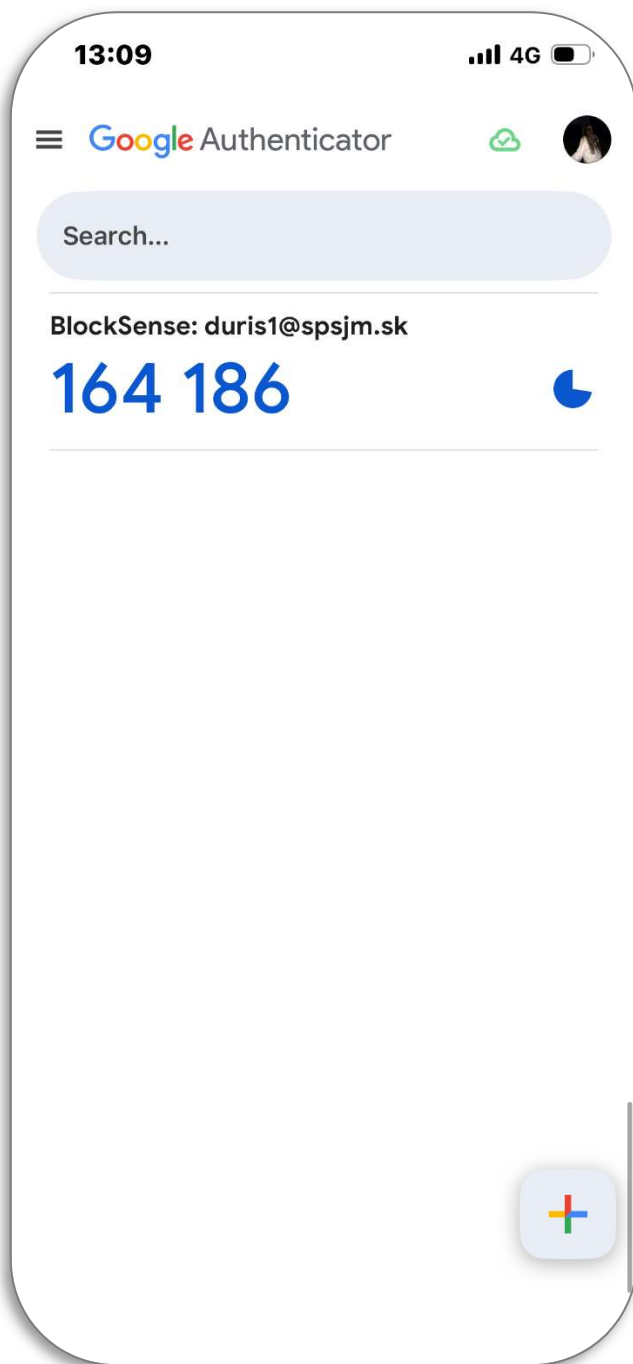
PRÍLOHA J KONFIGURÁCIA DVOJFAKTOROVEJ AUTENTIFIKÁCIE



PRÍLOHA K SPRÁVA DVOJFAKTOROVEJ AUTENTIFIKÁCIE



PRÍLOHA L GENEROVANIE TOTP KÓDU



PRÍLOHA M DIALÓG OVERENIA TOTP KÓDU

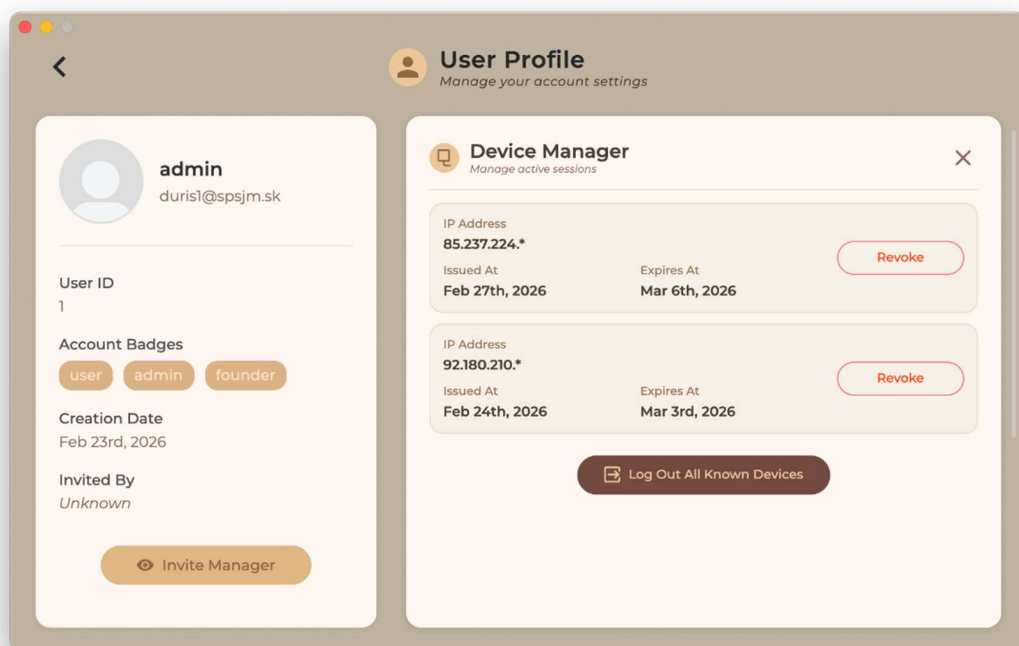


Enter Verification Code

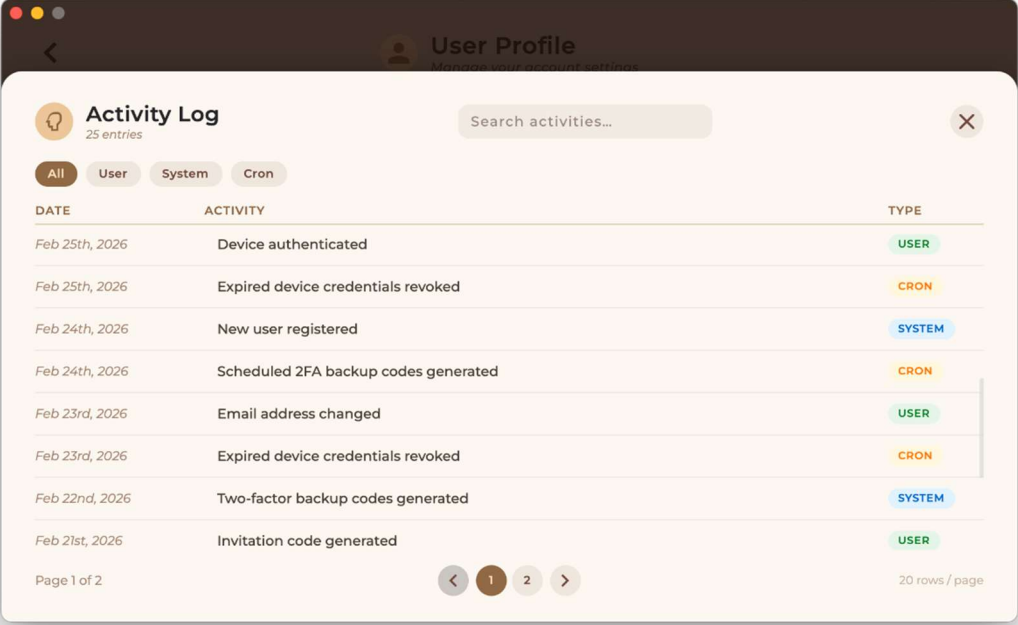
Please enter the 6-digit code from your authenticator app

[verify using backup code](#)

PRÍLOHA N SPRÁVA AKTÍVNYCH ZARIADENÍ



PRÍLOHA O ZOZNAM AKTIVITY POUŽÍVATEĽA



User Profile
Monitor your system's activities

Activity Log

25 entries

Search activities...

All User System Cron

DATE	ACTIVITY	TYPE
Feb 25th, 2026	Device authenticated	USER
Feb 25th, 2026	Expired device credentials revoked	CRON
Feb 24th, 2026	New user registered	SYSTEM
Feb 24th, 2026	Scheduled 2FA backup codes generated	CRON
Feb 23rd, 2026	Email address changed	USER
Feb 23rd, 2026	Expired device credentials revoked	CRON
Feb 22nd, 2026	Two-factor backup codes generated	SYSTEM
Feb 21st, 2026	Invitation code generated	USER

Page 1 of 2

< 1 2 >

20 rows / page

PRÍLOHA P SPRÁVA POZVÁNOK POUŽÍVATEĽA

Invitation Manager				
Search invites...				
Created	Expires	Invitation Code	Used By	Status
Dec 1st, 2025	Dec 31st, 2025	Click here to Expand	(not used)	Expired
Jan 13th, 2026	Feb 12th, 2026	Click here to Expand	(not used)	Revoked
Feb 27th, 2026	f0Fd5H2vUxkqxF62AeSKwzA4XH9I4XTv			Active
Jan 20th, 2026	Feb 19th, 2026	Click here to Expand	alice	Used

PRÍLOHA Q POTVRDENIE PIN KÓDU PEŇAŽENKY

Confirm Your PIN

Please re-enter your PIN to confirm

○ ○ ○ ○ ○ ○

Go Back **Confirm**

Press Enter or click Confirm when ready

(By pressing ESC you can return back)

PRÍLOHA R PROJEKTOVÉ SÚBORY

Vid' elektronická príloha

PRÍLOHA S FINÁLNY VÝSTUP PROJEKTU

Vid' elektronická príloha

PRÍLOHA T PREZENTAČNÉ VIDEO

Vid' elektronická príloha